AWQ:
Activation-aware Weight Quantization

Ji Lin, Jiaming Tang, et al. (MIT/Song Han Lab)

Presenter: Yuqi Liu

Motivation

Why deploy LLMs on edge devices?

- Eliminates cloud communication latency
- Reduces cloud infrastructure cost
- Enables offline operation
- Improves data privacy and security

The Challenge

- Modern LLMs are extremely large
- GPT-3: 175B parameters → ~350GB (FP16)
- Latest high-end GPU (B200): 192GB memory
- Edge devices have far less memory

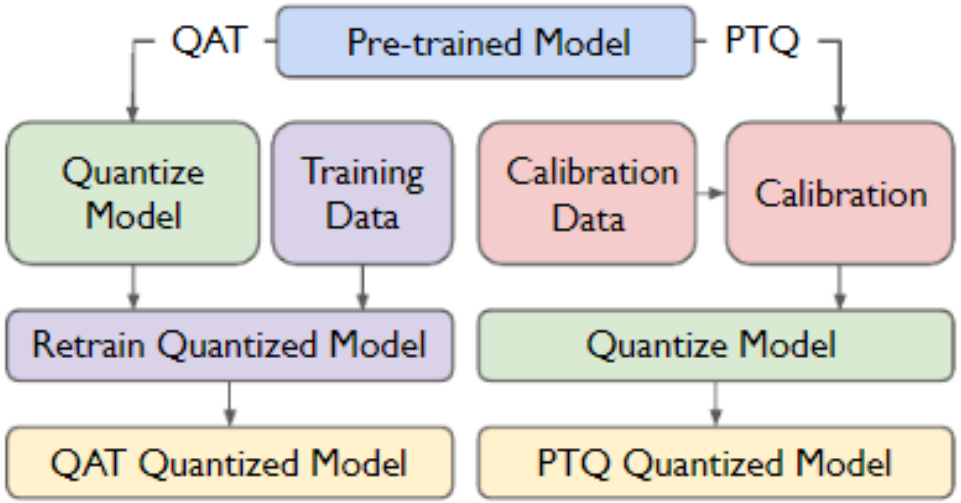
Problem Description

Quantization as a Solution

- Low-bit weight quantization reduces memory footprint
- Can significantly lower inference cost

Low-Bit Quantization Is Difficult

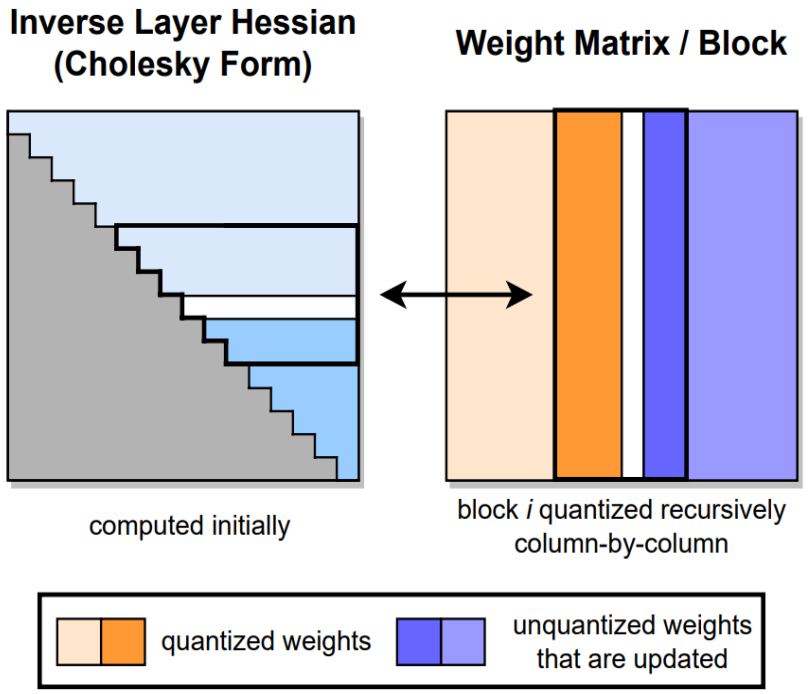
- **QAT (Quantization-Aware Training):**
  - Requires retraining
  - Too expensive for large LLMs
- **PTQ (Post-Training Quantization):**
  - No retraining
  - Suffers large accuracy degradation at low bits



Related Work: GPTQ

GPTQ
- Uses second-order error compensation
- Strong PTQ baseline

However:
- May overfit to calibration set
- Reconstruction can distort learned features
- Poor generalization to out-of-distribution domains



CORE OBSERVATION: NOT ALL WEIGHTS ARE EQUALLY IMPORTANT

- A very small fraction (≈0.1%–1%) of weight channels are **salient**
- Skipping quantization of only 0.1%–1% of weight channels significantly improves accuracy

PPL ↓	FP16	RTN (w3-g128)	FP16% (based on act.)			FP16% (based on W)			FP16% (random)		
			0.1%	1%	3%	0.1%	1%	3%	0.1%	1%	3%
OPT-1.3B	14.62	119.00	25.03	16.91	16.68	108.71	98.55	98.08	119.76	109.38	61.49
OPT-6.7B	10.86	23.54	11.58	11.39	11.36	23.41	22.37	22.45	23.54	24.23	24.22
OPT-13B	10.13	46.04	10.51	10.43	10.42	46.07	48.96	54.49	44.87	42.00	39.71

However, keeping the salient channels FP16 lead to mixed precision, making the system implementation difficult

New Solution:

Reduce their relative quantization error for important channels through scaling

Quantization Error

For a linear operation y = **wx**, the quantized version is:

$$y = Q(\mathbf{w})\mathbf{x} \tag{1}$$

Where the quantization function is defined as:

$$Q(\mathbf{w}) = \Delta \cdot \text{Round}(\frac{\mathbf{w}}{\Delta}), \quad \Delta = \frac{\max(|\mathbf{w}|)}{2^{N-1}} \tag{2}$$

Substitute into the linear operation, and the quantization error can be expressed as:

$$Q(w)x = \Delta \cdot \text{Round}\left(\frac{w}{\Delta}\right) x \tag{3}$$

$$\texttt{Err}(Q(w)x) = \Delta \cdot \texttt{RoundErr}(\frac{w}{\Delta}) \cdot x \tag{4}$$

Quantization Error

Substitute into the linear operation, and the quantization error can be expressed as:

$$Q(w)x = \Delta \cdot \text{Round}\left(\frac{w}{\Delta}\right) x \tag{3}$$

$$\texttt{Err}(Q(w)x) = \Delta \cdot \texttt{RoundErr}(\frac{w}{\Delta}) \cdot x \tag{4}$$

If the weight is scaled by s > 1:

$$Q(w \cdot s) \cdot \frac{x}{s} = \Delta' \cdot \text{Round}(\frac{ws}{\Delta'}) \cdot x \cdot \frac{1}{s} \tag{5}$$

$$\texttt{Err}(Q(w \cdot s)(\frac{x}{s})) = \Delta' \cdot \texttt{RoundErr}(\frac{ws}{\Delta'}) \cdot x \cdot \frac{1}{s} \tag{6}$$

The ratio of the new error to the original error is:

$$\frac{\Delta'}{\Delta} \cdot \frac{1}{s} \tag{7}$$

Quantization Error

OPT-6.7B	$s = 1$	$s = 1.25$	$s = 1.5$	$s = 2$	$s = 4$
proportion of $\Delta' \neq \Delta$	0%	2.8%	4.4%	8.2%	21.2%
average Δ'/Δ	1	1.005	1.013	1.038	1.213
average $\frac{\Delta'}{\Delta} \cdot \frac{1}{s}$	1	0.804	0.676	0.519	**0.303**
Wiki-2 PPL	23.54	12.87	12.48	**11.92**	12.36

- For OPT-6.7B, perplexity improves from **23.54 (no scaling)** to **11.92 (s = 2)**.
- Increasing s reduces quantization error for salient channels, best performance achieved at s = 2.
- Further increasing will increase the quantization error for non-salient channels

Searching for the Optimal Scaling Factor

$$\mathbf{s}^* = \arg\min_{\mathbf{s}} \mathcal{L}(\mathbf{s})$$

$$\mathcal{L}(\mathbf{s}) = \|Q(\mathbf{W} \cdot \mathrm{diag}(\mathbf{s}))(\mathrm{diag}(\mathbf{s})^{-1} \cdot \mathbf{X}) - \mathbf{W}\mathbf{X}\|$$

Choose per-channel scaling s that minimizes the difference between the quantized layer output and the original output.

s is high-dimensional, Q is not differentiable (rounding), so standard gradient descent cannot be used, and approximate gradient methods are unstable

Searching for the Optimal Scaling Factor

Reduce the search space based on the insight that Channel importance is correlated with activation magnitude

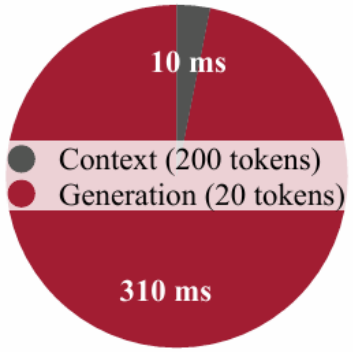
$$\mathbf{s} = \mathbf{s_X}^{\alpha}$$

sX : the average magnitude of activation (per-channel)
α : balance between the protection of salient and non-salient channels, $0 < \alpha < 1$

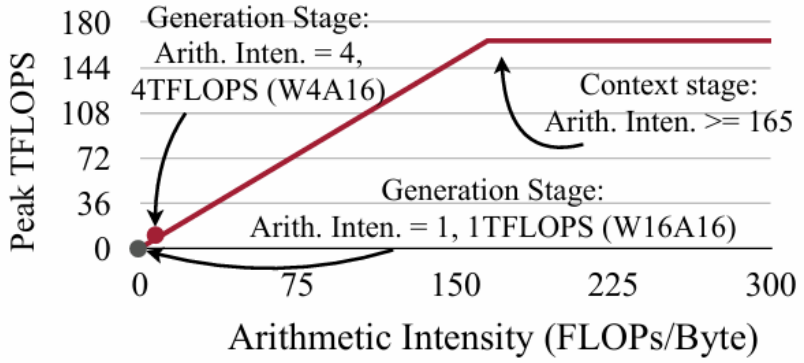
$$\alpha^* = \arg\min_{\alpha} \mathcal{L}(\mathbf{s_X}^{\alpha})$$

Performs a grid search for α, compute the loss on a small calibration dataset.

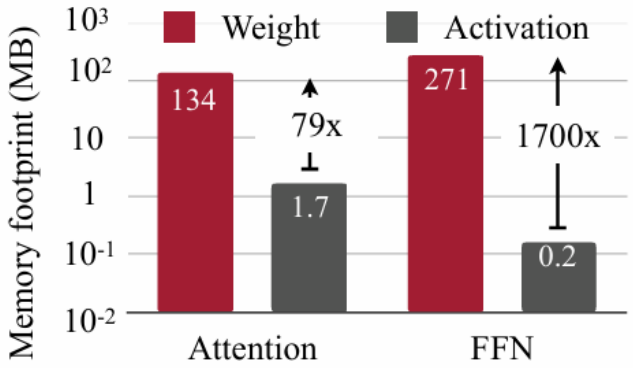
Why AWQ Helps Accelerate On-Device LLMs



(a) Generation stage is slower



(b) Generation stage is bounded by memory bandwidth



(c) Weight loading is more expensive

- Generation dominates latency for interactive use
- Generation is **strongly memory-bound**
- To speed up generation, we need to reduce memory traffic.
- Most memory traffic comes from loading weights.
- For W4A16, weight memory traffic reduces by 4×

From Memory Reduction to Real Speedup

W8A8 vs W4A16

- **W8A8:** Same precision for storage & compute → easier kernel design
- **W4A16:** Different precision for storage & compute → requires on-the-fly dequantization

$$Q(w)x = \Delta \cdot \text{Round}\left(\frac{w}{\Delta}\right) x$$

Memory Saving ≠ Automatic Speedup

- LLM inference is memory-bound
- Dequantization overhead can offset gains

TinyChat

On-the-fly dequantization fused with GEMM

- Convert INT4 → FP16 inside matrix multiply
- Avoid writing dequantized weights to memory

Hardware-aware weight packing

- SIMD-friendly layout
- Reduce unpacking overhead

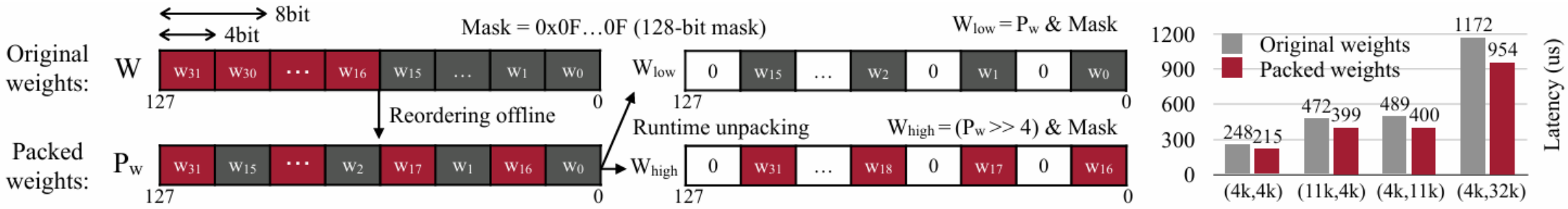


Figure 4. SIMD-aware weight packing for ARM NEON with 128-bit SIMD units. Original weights are reordered and packed to align with the bit width so that the weights can be unpacked into bytes at runtime using AND and shift bitwise operations with a 128-bit mask.

Experimental Setup

Quantization Setup

- Weight-only grouped quantization
- Group size = 128
- Focus on INT3 / INT4
- AWQ uses small calibration set
- Grid search (20 steps) for optimal α

Models Evaluated

- LLaMA and LLaMA-2
- OPT family
- Instruction-tuned Vicuna
- Vision-language models: OpenFlamingo-9B, LLaVA-13B

Evaluation Metric

- Perplexity on WikiText-2

Baselines Compared

- RTN (Round-to-Nearest)
- GPTQ

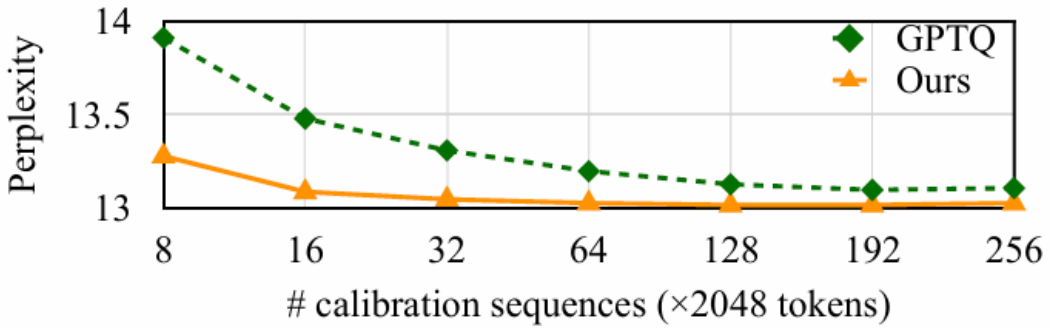
Results: LLaMA and Mistral

PPL↓		Llama-2			LLaMA			
		7B	13B	70B	7B	13B	30B	65B
FP16	-	5.47	4.88	3.32	5.68	5.09	4.10	3.53
INT3 g128	RTN	6.66	5.52	3.98	7.01	5.88	4.88	4.24
	GPTQ	6.43	5.48	3.88	8.81	5.66	4.88	4.17
	GPTQ-R	6.42	5.41	3.86	6.53	5.64	4.74	4.21
	AWQ	**6.24**	**5.32**	**3.74**	**6.35**	**5.52**	**4.61**	**3.95**
INT4 g128	RTN	5.73	4.98	3.46	5.96	5.25	4.23	3.67
	GPTQ	5.69	4.98	3.42	6.22	5.23	4.24	3.66
	GPTQ-R	5.63	4.99	3.43	5.83	5.20	4.22	3.66
	AWQ	**5.60**	**4.97**	**3.41**	**5.78**	**5.19**	**4.21**	**3.62**

Table 4. AWQ improves over round-to-nearest quantization (RTN) for different model sizes and different bit-precisions. It consistently achieves better perplexity than GPTQ (w/ and w/o reordering) on LLaMA & Llama-2 models.

Wikitext2 PPL↓	Mixtral-8x7B	Mistral-7B
FP16	5.94	4.14
INT4-g128	6.05	4.30
INT3-g128	6.52	4.83

Results: Data Efficiency and Generalization



(a) Our method needs a smaller calibration set

Calib \ Eval	GPTQ			Ours		
	PubMed	Enron		PubMed	Enron	
PubMed	32.48	50.41	+4.89	32.56	45.07	+0.50
Enron	+2.33 34.81	45.52		+0.60 33.16	44.57	

(b) Our method is more robust to calibration set distribution

Figure 8. Left: AWQ needs a much smaller calibration set to reach a good quantized performance. It can achieve better perplexity using 10× smaller calibration set compared to GPTQ. **Right:** Our method is more robust to the calibration set distribution. Overall, using the same calibration and evaluation distribution works the best (PubMed-PubMed, Enron-Enron). But when using a different calibration distribution (PubMed-Enron, Enron-PubMed), AWQ only increases the perplexity by 0.5-0.6, while GPTQ has 2.3-4.9 worse perplexity. All experiments are done with the OPT-6.7B model under INT3-g128 quantization.

Strengths & Future Improvements

Strengths

- Simple and intuitive insight (activation-aware scaling)
- No retraining required (efficient PTQ)
- Hardware-friendly (uniform INT4, no mixed precision)
- Strong empirical performance and generalization
- Real system speedup via TinyChat (3×+)

Potential improvements

- Develop stronger theoretical understanding of the scaling mechanism
- Extend beyond weight-only quantization (e.g., activation quantization)
- Improve robustness at ultra-low precision (e.g., INT2)

Results: Speedup

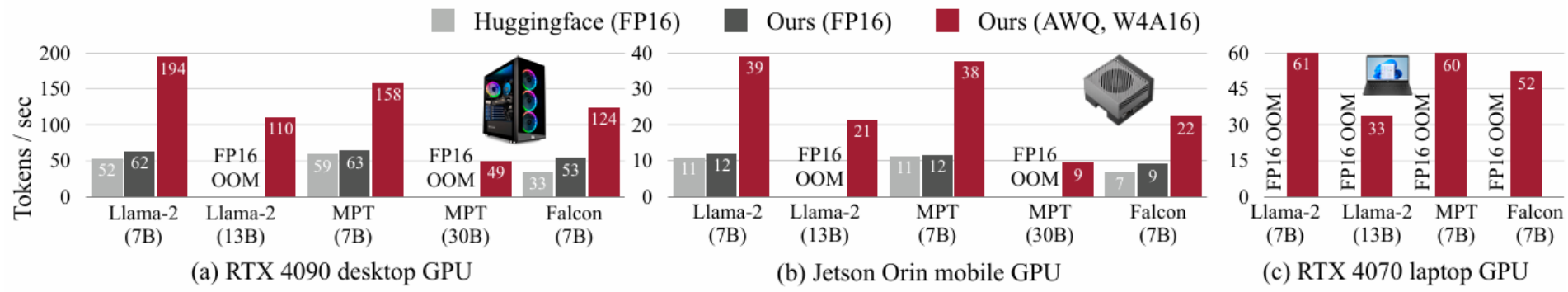


Figure 9. TinyChat provides a turn-key solution to transform the theoretical memory footprint reduction into a quantifiable speedup. As a result, TinyChat is up to **3.9**× and **3.5**× faster than the FP16 implementation from Huggingface on 4090 (desktop GPU) and Orin (mobile GPU), respectively. AWQ also democratizes Llama-2-13B deployment on laptop GPUs (4070) with merely 8GB memory.

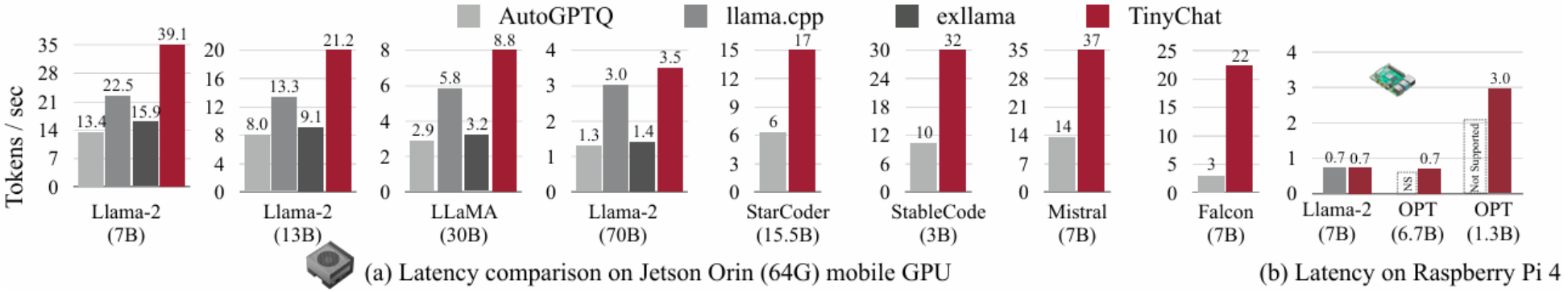


Figure 10. TinyChat offers **1.2-3.0**× speedup over existing systems when running 4-bit quantized Llama models on NVIDIA Jetson Orin. It also supports a diverse range of general-purpose and coding-specific LLMs with at least **2.6**× speedup over AutoGPTQ, which also supports all these workloads. Moreover, TinyChat seamlessly operates on Raspberry Pi and enables the deployment of LLMs with up to 7 billion parameters on extremely resource-constrained IoT devices.

Radial Attention: $\mathcal{O}(n \log n)$ Sparse Attention with Energy Decay for Long Video Generation

Xingyang Li, Muyang Li, Tianle Cai, et al.

MIT NVIDIA Princeton UC Berkeley Stanford

NeurIPS 2025

Outline

The Bottleneck: Quadratic Attention in Video Diffusion

Video diffusion models use 3D dense self-attention:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V}$$

The problem:

- A 5s 720p video $\approx$ **110K tokens**
- Dense attention scales $\mathcal{O}(n^2)$ — cost explodes with length
- Training and inference on long videos is prohibitive

Goal: Efficient attention that

- Scales sub-quadratically with video length
- Preserves video quality
- Works for both training and inference

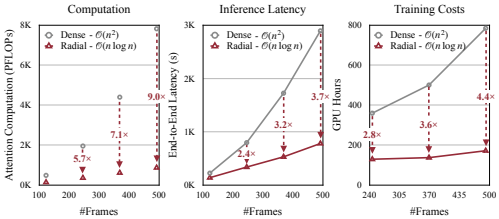


Figure: Radial Attention reduces attention from $\mathcal{O}(n^2)$ to $\mathcal{O}(n \log n)$. 9× attention reduction at 509 frames.

Why Existing Solutions Fall Short

Method	Approach	Limitation
SVG	Dynamic spatial/temporal head classification	Runtime profiling errors; not for training
STA	3D sliding window attention	Fixed receptive field; misses long-range deps
Linear Attn	Replace softmax with linear kernel	Loses local detail; needs full retraining
PowerAttn	Power-of-two distances ($\mathcal{O}(n \log n)$)	Ignores spatial structure in video
RIFLEx	RoPE frequency adjustment	Quality degrades beyond $2\times$

Key Insight from Physics

"Nothing spreads without loss; every signal, every influence, every attention — decays with distance."

Just as physical signals lose energy over distance, **attention scores decay exponentially** with spatiotemporal distance between tokens.

Related Works

Video Diffusion Models

- 2D UNet + temporal modules $\rightarrow$ DiT backbone $\rightarrow$ **3D dense attention** (current SOTA)
- Wan2.1-14B, HunyuanVideo, Mochi 1, CogVideoX

Efficient Attention for Video

- **SVG**: dynamic head profiling (spatial or temporal per head)
- **STA**: fixed 3D sliding window
- **SageAttention**: quantized attention kernels

LLM Sparse Attention $\rightarrow$ Video

- **LongLoRA**: shifted local windows
- **PowerAttention**: power-of-two distances
- Both ignore video's spatial structure

Long Video Generation

- **RIFLEx**: training-free via RoPE (limited to $2\times$)
- **FramePack**: autoregressive short clips
- **Linear attention**: $\mathcal{O}(n)$ but loses fidelity

Prior $\mathcal{O}(n \log n)$ Attention

- Reformer, H-Transformer-1D — hardware-unfriendly

Overview: SVG vs. Radial Attention

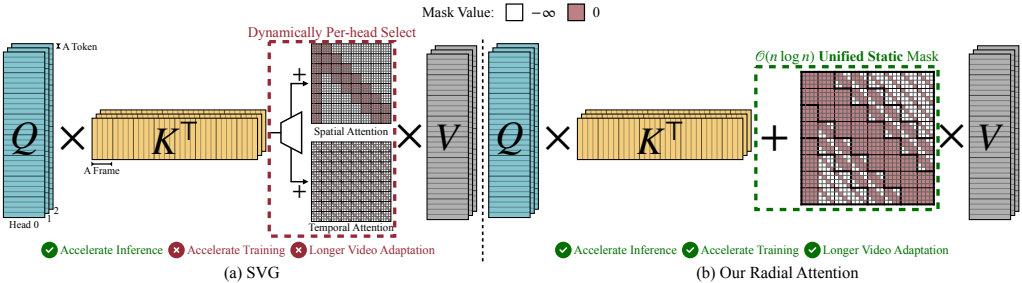
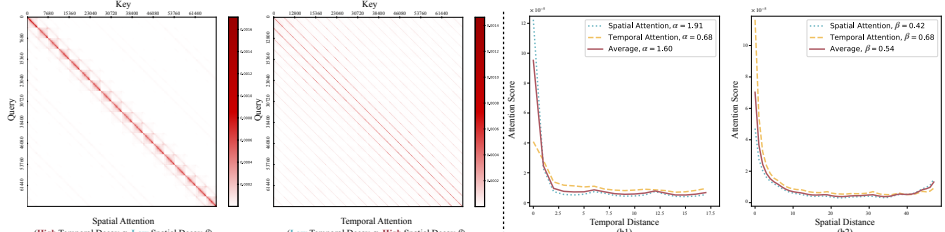


Figure: **(a)** SVG dynamically selects spatial or temporal attention per head — cannot train on longer videos. **(b)** Radial Attention uses a *static* mask that unifies both with $\mathcal{O}(n \log n)$ complexity, enabling efficient length extension.

Spatiotemporal Energy Decay



(a) Attention Map (b) Attention Score Distribution

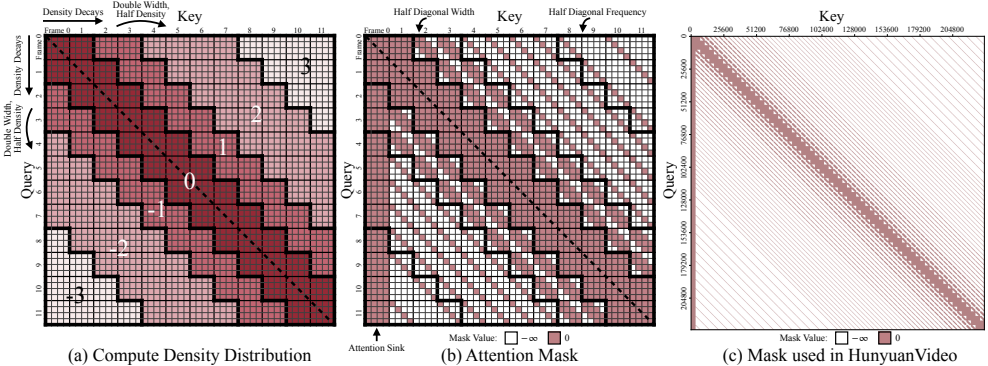
Observation: Post-softmax attention scores decay exponentially with spatial and temporal distance.

For query at frame i_0, position k_0:

$$p_{js+l} \leq C_{\text{rel}}\, e^{-\alpha|j-i_0|-\beta|l-k_0|}\, p_{i_0 s+k_0}$$

- α: temporal decay rate
- β: spatial decay rate
- Exponential regression: $R^2 > 0.985$
- Unifies spatial & temporal heads

Radial Attention: Mask Design



(a) Compute Density Distribution (b) Attention Mask (c) Mask used in HunyuanVideo

Temporal density decay:

- Density: $\left(\frac{1}{2}\right)^{\lfloor \log_2(\max(|i-j|,1)) \rfloor}$
- Band 0: full density
- Each outer band: half density, double width
- $\Rightarrow$ constant compute per band

Spatial density decay:

- Diagonal width: $\lfloor s/2^{\lfloor \log_2 \max(|i-j|,1) \rfloor} \rfloor$
- Close frames: wide diagonal
- Distant frames: narrow / subsampled
- Attention sink on first frame

Radial Attention: Formal Definition & Complexity

4D attention mask $\tilde{\mathbf{M}} \in \{-\infty, 0\}^{f \times f \times s \times s}$:

$$\tilde{M}_{i,j,k,l} = \begin{cases} 0, & \text{if } 2^{\lfloor \log_2 \max(|i-j|,1) \rfloor} \le s \ \text{ and } \ |k-l| + 1 \le \dfrac{s}{2^{\lfloor \log_2 \max(|i-j|,1) \rfloor}} \\ 0, & \text{if } |i-j| \bmod \lceil 2^{\lfloor \log_2 \max(|i-j|,1) \rfloor} / s \rceil = 0 \ \text{ and } \ k = l \\ -\infty, & \text{otherwise} \end{cases}$$

Complexity: $\mathcal{O}(n \log n)$

$$\#\text{zeros} \le 4s^2 f \log_2 f = 4sn(\log_2 n - \log_2 s)$$

For fixed resolution s, scales as $\mathcal{O}(n \log n)$ with video length.

Error Bound

$$\|\tilde{p} - p\|_1 = O\Big(C_{\text{rel}}\, e^{-\min(\beta/2,\, \alpha)\, s}\Big)$$

Error decreases **exponentially** with resolution s and decay rates α, β.

LoRA Fine-Tuning for Long Videos

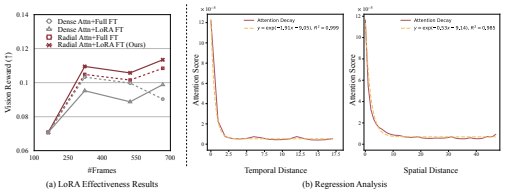
Why LoRA works well with Radial Attention:

- Radial Attention only prunes low-energy pairs, **preserves softmax**
- Pre-trained weights remain largely valid
- Only minimal adaptation needed

Setup:

- LoRA (rank 128) on Q, K, V, O projections
- 2K curated videos from OpenVid-1M per target length
- 8× H100, 16–21 hours (HunyuanVideo)
- Dense attention on first 2 DiT blocks + 12 warmup steps

Key finding: LoRA + Radial Attention **matches or outperforms** full fine-tuning — focuses updates on the most critical parameters



(a) LoRA Effectiveness Results (b) Regression Analysis

Figure: **(a)** LoRA + Radial Attention outperforms full fine-tuning with dense attn. **(b)** Exponential fit: $R^2 > 0.985$.

Experimental Setup

Models:

- Wan2.1-14B (14B params, 81 frames, 720p)
- HunyuanVideo (13B params, 125 frames, 720p)
- Mochi 1 (10B params, 162 frames, 480p)

Metrics:

- Vision Reward (human rating proxy)
- PSNR, SSIM, LPIPS (similarity to dense)
- VBench-long (subject consistency, aesthetic quality, imaging quality)

Baselines:

- SVG (dynamic sparse)
- STA (sliding window, FA3)
- PowerAttention ($\mathcal{O}(n \log n)$ for LLMs)
- LongLoRA (shifted local)
- SANA (linear attention)
- RIFLEx (training-free)

Hardware: NVIDIA H100 GPUs

- Default length: single GPU
- Long videos: 8 GPUs

Results: Default Video Length (Training-Free)

Model	Method	PSNR↑	SSIM↑	LPIPS↓	PFLOPs	Speedup
HunyuanVideo (117 frames)	Original	–	–	–	612	–
	STA (FA3)	26.7	0.866	0.167	331	2.29×
	PowerAttn	22.1	0.764	0.256	339	1.65×
	SVG	27.2	**0.895**	**0.114**	340	1.90×
	Ours	**27.3**	0.886	**0.114**	339	1.88×
Wan2.1-14B (69 frames)	Original	–	–	–	560	–
	STA (FA3)	22.9	0.830	0.171	322	2.01×
	PowerAttn	22.4	0.790	0.176	324	1.67×
	SVG	23.2	0.825	0.202	324	1.71×
	Ours	**23.9**	**0.842**	**0.163**	323	1.77×

- Under **same compute budget**: matches SVG quality with a *static* mask (no profiling)
- Consistently outperforms STA and PowerAttention on all similarity metrics
- Up to **1.9× end-to-end speedup** on a single H100

Visual Comparison: Default Video Length

Figure: Radial Attention preserves high fidelity compared to the original dense attention, while STA shows visible quality degradation. Results on HunyuanVideo (117 frames, 768p) and Wan2.1-14B (69 frames, 768p).

Results: Long Video Generation (4× Length)

Model	Method	Sparsity	Train Speedup	Infer Speedup	Vision Reward↑	S.C.↑
HunyuanVideo (509 frames)	Original	0%	–	1.0×	0.054	0.988
	RIFLEx	0%	–	1.0×	0.037	0.989
	Full FT	0%	1.0×	1.0×	0.133	0.977
	LongLoRA	88.4%	4.48×	3.61×	0.130	0.936
	Ours	88.3%	**4.37**×	**3.71**×	**0.134**	0.973
Mochi 1 (667 frames)	Original	0%	–	1.0×	−0.091	0.916
	Full FT	0%	1.0×	1.0×	0.099	0.934
	PowerAttn	86.5%	2.84×	2.60×	0.107	0.956
	Ours	85.5%	**2.83**×	**2.57**×	**0.113**	**0.958**

Efficiency Gains

- Up to **4.4× training speedup**
- Up to **3.7× inference speedup**
- **88% sparsity** at 4× length

Quality Preserved

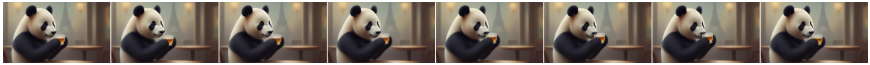
- Matches dense full fine-tuning quality
- SANA (linear attn) fails catastrophically
- RIFLEx degrades beyond 2×

Visual Comparison: 4× Longer Videos

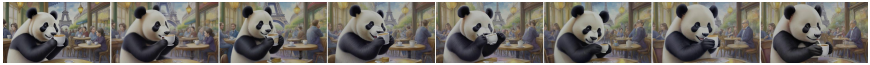
Prompt: A gentle and curious panda, with soft, fluffy fur and large round eyes, is depicted in a charming watercolor painting. The panda sits at a cozy table in a quaint cafe located in the heart of Paris.

Original HunyuanVideo, VisionReward: 0.055, Latency: 2895s, Inference Speedup: 1.0×, Training Time: 0 GPU Hours

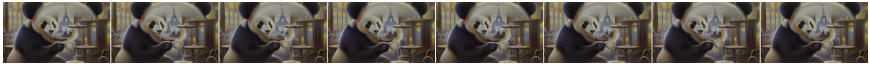
HunyuanVideo+RIFLEx, VisionReward: 0.055, Latency: 2895s, Inference Speedup: 1.0×, Training Time: 0 GPU Hours



HunyuanVideo+LoRA+Spatial Head, VisionReward: 0.121, Latency: 755s, Inference Speedup: 3.8×, Training Time: 160 GPU Hours, Training Speedup: 4.5×

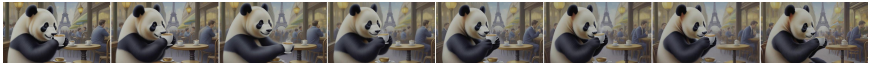


HunyuanVideo+LoRA+Temporal Head, VisionReward: 0.021, Latency: 774s, Inference Speedup: 3.7×, Training Time: 169 GPU Hours, Training Speedup: 4.4×



HunyuanVideo+LongLoRA, VisionReward: 0.122, Latency: 803s, Inference Speedup: 3.6×, Training Time: 168 GPU Hours, Training Speedup: 4.5×

HunyuanVideo+LoRA+PowerAttention, VisionReward: 0.120, Latency: 766s, Inference Speedup: 3.8×, Training Time: 174 GPU Hours, Training Speedup: 4.3×



Ablation Studies

Attention Error (MSE):

Method	Attn MSE ↓
STA	1.5×10^{-2}
SVG	4.4×10^{-3}
Ours	$\mathbf{3.9 \times 10^{-3}}$

Mask pattern comparison:

- Radial (exponential decay) outperforms Harmonic Series (1/distance) on all metrics

Design choices:

Choice	Optimal
Dense warmup steps	12 (default), 2 (long)
Dense initial layers	2 DiT blocks
LoRA rank	128

LoRA compatibility:

- Length-extension LoRA merges with existing **style LoRAs** seamlessly
- Works at both default and extended lengths

Teaser: Long Video Generation Results

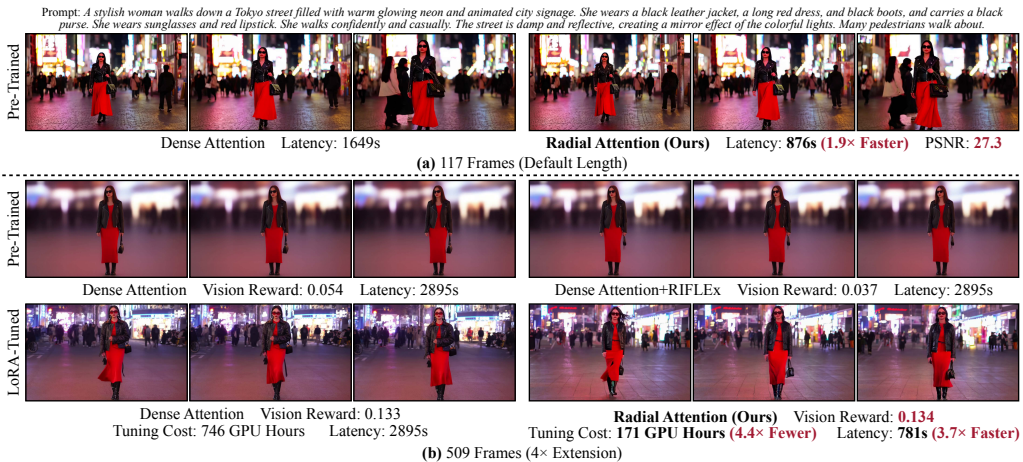


Figure: Radial Attention accelerates HunyuanVideo by 1.9× at default length. For 4× longer videos: 4.4× training cost reduction, 3.7× inference speedup.

Discussion: Strengths & Limitations

Key Contributions

1. **Spatiotemporal Energy Decay**: formalized exponential decay of attention in video diffusion
2. **Radial Attention**: static $\mathcal{O}(n \log n)$ sparse mask unifying spatial & temporal attention
3. **LoRA-based length extension**: $4\times$ longer video at a fraction of cost

Advantages

- Static mask: no profiling, works for training
- Hardware-friendly 128×128 block sparsity
- Preserves softmax $\Rightarrow$ easy to adapt

Limitations

- Still **quadratic in resolution** s — only $\mathcal{O}(n \log n)$ when frames grow at fixed s
- Exponential decay is a **simplification** — real dependencies are more complex
- Requires dense warmup (first 12 steps, first 2 blocks)

Future Directions

- Native sparse attention pre-training (NSA, MoBA)
- Address resolution scaling
- Larger LoRA training datasets
- FlashAttention-3 integration

Summary

Radial Attention translates the physical principle of **spatiotemporal energy decay** into a static sparse attention mask with $\mathcal{O}(n \log n)$ complexity.

Default Length	**4× Training**	**4× Inference**
$1.9\times$	$4.4\times$	$3.7\times$
speedup	cheaper	faster
(no quality loss)	(LoRA fine-tuning)	(quality preserved)

Code: `https://github.com/mit-han-lab/radial-attention`